Work-in-Progress: Towards Real-Time IDS via RNN and Programmable Switches Co-Designed Approach

Ziming Zhao†, Zhaoxuan Li‡, Zhuoxue Song†, Fan Zhang†
†Zhejiang University, Hangzhou, 310027, China
‡SKLOIS, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, 100093, China

***Abstract*—Existing Deep Learning (DL)-based network Intrusion Detection System (IDS) is able to characterize sequence semantics of traffic and discover malicious behaviors. Yet DL models are often nonlinear and highly non-convex functions that are difficult for in-network real-time deployment, *i.e.,* existing DL solutions are essentially offline analysis. In this paper, we present RIDS, a hardware-friendly Recurrent Neural Network (RNN) model that is co-designed with programmable switches. As its core, RIDS is powered by two tightly-coupled components: (i) rLearner, the RNN learning module with in-network deployability as the first-class requirement; and (ii) rEnforcer, the concrete pipeline design to realize rLearner-generated models inside the network dataplane. We implement a prototype of RIDS and evaluate it on our physical testbed. The experiments show that RIDS could satisfy both detection performance and high-speed bandwidth adaptation simultaneously, when none of the other existing approaches could do so. Inspiringly, RIDS realizes remarkable intrusion/malware detection effect (*e.g.,* ∼99% F1 score) and model deployment (*e.g.,* 100 Gbps per port), while only imposing nanoseconds of latency.**

***Index Terms*—Intrusion/malware detection system, hardware-friendly deep learning, programmable switches**

I. Introduction

Intrusion Detection Systems (IDS) is one of the critical security mechanisms for detecting on-going malicious activities in networked systems. Over the past decades, the proposed approaches gradually evolve from policy-based [14] to deep learning-based (DL-based) detection [13] in the IDS landscape. The former aims to describe the special pattern for each type of attack, so it depends on the knowledge completeness of attack patterns (or for whitelists, prior knowledge of legitimate traffic is required). The latter has superior generalization capability [10], especially being able to extract the temporal semantics of traffic such as Recurrent Neural Network (RNN) [8]. However, existing DL models do not perform line-speed detection in the data plane, essentially offline analysis.

We outline eight representative IDS and draw the detection method (development trend) & throughput in Figure 1. Existing detection systems either perform limited model capability or cannot adapt to high-speed bandwidth. For example,

Corresponding author: Fan Zhang (fanzhang@zju.edu.cn). This work is supported in part by National Natural Science Foundation of China (62227805, 62072398), by SUTD-ZJU IDEA Grant for visiting professors (SUTD-ZJUVP201901), by National Key R&D Program of China (2020AAA0107700), by National Key Laboratory of Science and Technology on Information System Security (6142111210301), by State Key Laboratory of Mathematical Engineering and Advanced Computing, and by Key Laboratory of Cyberspace Situation Awareness of Henan Province (HNTS2022001).

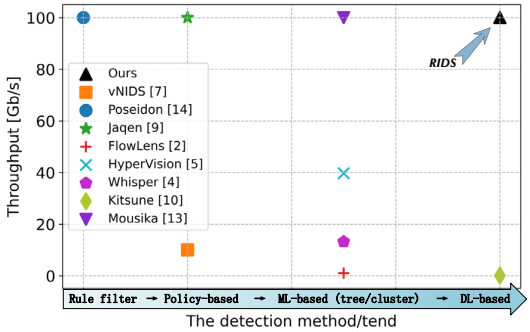

Fig. 1: The detection method/tend & throughput diagram.

Mousika [13] deploys the decision tree in the data plane to guarantee line-speed processing, but cannot maintain flow-level information. Regarding Deep Neural Network (DNN) methods, Kitsune [10] puts effort into making AutoEncoder-based detection run efficiently, while it only supports ∼112 Mb/s throughput, still has a big gap with modern 100 Gbps bandwidth. **Requirement.** Considering the importance of real-time detection in safety-critical scenarios [10], [13], [14] (*e.g.,* practitioners can timely disconnect malicious sessions according to the detection results), we tend to implement both complex model capability and high bandwidth adaptation, *i.e.,* achieving the black triangle in the upper right part of Figure 1.

With developments of Application Specific Integrated Circuit (ASIC), recently emerging programmable switches allow user-customized packet processing logic in a protocol-independent manner. It motivates us to design hardware-friendly DNN inference to embrace this progress. Towards this end, we recognize two major challenges in realizing on-chip intelligent traffic analysis. (i) Offloading feature extraction and flow state maintenance to the switches. Typical feature extraction, without strict time requirements, tends to be comprehensive and requires complex calculations. Thus, the first step is to realize line-speed feature extraction and flow state maintenance. (ii) Achieving tailor-made model inference to cope with hardware resource constraints. A more crucial problem is to enable on-chip points to process packets based on the inference results. DNN models are either nonlinear and highly non-convex functions or involve lots of multiplication and activation operations, whereas programmable switches usually have limited computation power, *e.g.,* do not support multiplication and floating number computations. So designing the hardware-friendly DL models inside the switches is non-trivial.

2023 IEEE Real-Time Systems Symposium (RTSS) | 979-8-3503-2857-8/23/$31.00 ©2023 IEEE | DOI: 10.1109/RTSS59052.2023.00045

2576-3172/23/$31.00 ©2023 IEEE
DOI 10.1109/RTSS59052.2023.00045

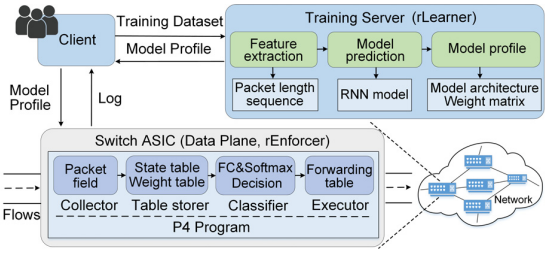

Fig. 2: RIDS architecture and components.

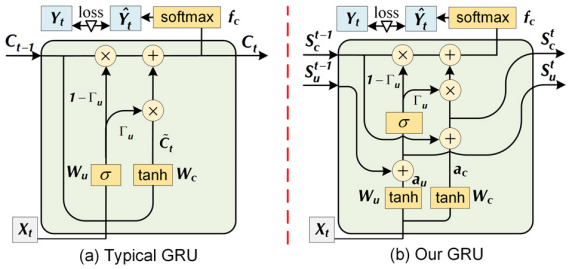

Fig. 3: Traditional GRU architecture and ours.

In this paper, we present RIDS (RNN-based IDS, for short), a co-designed system achieving DL-driven intrusion detections at network line speed by leveraging the emerging hardware primitive. Our **contributions** are summarized following:

- We design the hardware-friendly inference processing for the RNN model, which only requires bitwise operations or integer additions/subtractions, including ternary weights, binary activation functions, and decoupled stateful tables.
- We leverage the emerging programmable switches primitives and architect the tailor-made RNN model as a series of dataplane modules to realize the line-speed detection.
- We implement a prototype of RIDS on our physical testbed, and conduct extensive evaluations to demonstrate the advantages of RIDS. The experiments show that RIDS realizes >99% F1 score for intrusion/malware detections, while only imposing negligible overhead, *e.g.,* nanoseconds of latency on a 100 Gbps bandwidth.

II. Background and Motivation

Existing intrusion detection systems. Existing intrusion detection systems can be roughly categorized into policy-based, machine learning-based (ML-based), and deep learning-based (DL-based) approaches. (i) *Policy-based* methods [7], [9], [14] aim to profile the special pattern and set up primitives accordingly for each attack type. The effectiveness of these approaches largely depends on prior knowledge about attack signatures. (ii) Unlike policy-based approaches, *ML-based* IDS performs feature extraction and data fitting. Some clustering-based methods [4], [5] are deployed in Intel's Data Plane Development Kit (DPDK) [11] to support about 10∼50 Gbps throughput and introduce a latency of a few seconds. Some decision tree (DT)-based methods combine with programmable switches, FlowLens [2] realizes feature extraction on the data plane to make only ∼1 Gbps bandwidth being saturated. Mousika [13] converts DT into a series of *if-else* branches while cannot maintain flow-level states. (iii) *DL-based* approaches embrace the advances in deep neural networks to identify attack traffic. Kitsune [10] is the deployable DL state-of-the-art (SOTA) model that uses AutoEncoder to detect abnormal traffic, but only can support ∼112 Mbps.

Opportunities and constraints of programmable switches. Dataplane programmability is the recent trend in networking innovations. (i) *Opportunities.* In contrast to OpenFlow-based software-defined networking, P4-programmable networks provide new features that can be reconfigured directly in hardware [3]. In programmable switches, each hardware stage is integrated with Arithmetic Logic Units (ALUs) that can perform computations at line speed, making it possible to implement terabit packet processing devices. Operators could use domain-specific languages (*e.g.,* P4 [3]) to customize the dataplane logic. (ii) *Constraints.* However, the new primitives of programmable switches have limitations. Switch ASIC contains a very small amount of memory and only a fraction of the total available SRAM (Static Random-Access Memory) can be used to allocate register arrays. There also are other constraints due to the special memory architecture of switching ASICs. For example, we cannot access the registers of one stage from other stages. To ensure line-speed packet forwarding, the switches restrict the total number and type of operations allowed within each stage. Specifically, multiplications, divisions or floating-point operations, and variable-length loops are not supported. And each action on tables can only perform a restricted set of simple operations, such as additions, bitwise shifts, and memory accesses [2]. RIDS is therefore designed to fulfill on-switch DL inference despite these constraints.

III. Design of RIDS

In Figure 2, we depict a high-level architecture of RIDS, including the key building blocks of each component. Among them, rLearner is responsible for training an RNN model that only involves bitwise operations, integer addition/subtraction, and conditional branches. rEnforcer is the actual component that runs on rLearner-produced models.

A. rLearner Design

rLearner is an RNN learning module designed with deployability as a first-class requirement. Thus, rLearner focuses on obtaining a hardware-friendly model whose inference process can be only based on readily available primitives on ASICs, such as bitwise operations or integer additions/subtractions.

1) Feature Selection and Processing: Considering that prior arts have demonstrated that the sequence of packet length is a valid feature as the fingerprint of flow [2], [8], so we choose the packet length sequence as the per-flow feature. To avoid numerical overflow, we also execute one-hot encoding on the packet length, referring to four intervals $[0, 64), [64, 128), [128, 256), [256, \infty)$.

2) Hardware-Friendly Model Inference: The second design challenge is how to realize hardware-friendly RNN inference. **Matrix calculation.** Matrix multiplication is the main part of model inference. We leverage ternarized matrix [6] (+1, 0

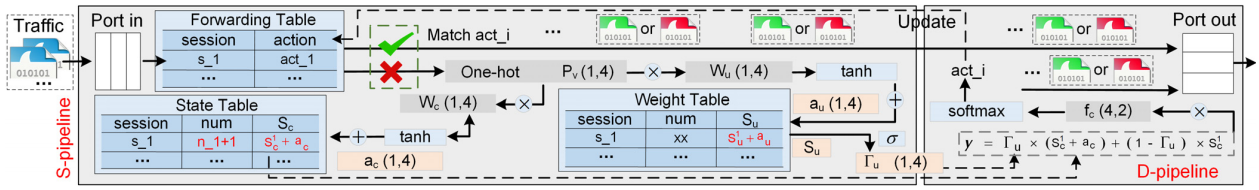

Fig. 4: The architecture of the rEnforcer scheduler designed to deploy RNN models in dataplane.

or -1) to transform multiplication into additions/subtractions. Specifically, the ternarization operation refers to

$$W_i^t = f_t\left(W_i|\Delta\right)\begin{cases} +1, & if\ W_i > \Delta, \\ 0, & if\ |W_i| \le \Delta, \\ -1, & if\ W_i < -\Delta. \end{cases}$$

where Δ is a positive threshold parameter.

Activation function. We use the following formulas to realize weight activation (binarized *sigmoid*) and state activation (binarized *tanh*), where λ_1 and λ_2 are learnable parameters.

$$\sigma(x) = \begin{cases} +1, & if\ x \ge \lambda_1, \\ 0, & otherwise. \end{cases} \quad tanh(x) = \begin{cases} +1, & if\ x \ge \lambda_2, \\ -1, & otherwise. \end{cases}$$

3) Store Per-Flow State: In programmable switches, variable operations between *read-write* must be completed in one stage (*e.g.,* only one addition calculation), and a variable can only be accessed once when forwarding an individual packet. Due to these constraints, the per-flow state cannot be stored as the traditional RNN does. To this end, we decouple the hidden state C_{t-1} and the input X_t. Thus the set matrix W only multiplies with X_t. The results of $W \cdot X_t$ can be used to update the hidden state and weight element-wisely. Specifically, for the t-th time step, we can update the state table and weight table that only involves one addition calculation, as shown in Figure 3(b).

$$\begin{cases} a_c = \tanh\left(W_c \cdot X_t + b_c\right) \\ a_u = \tanh\left(W_u \cdot X_t + b_u\right) \end{cases} \begin{cases} S_c^t = S_c^{t-1} + a_c \\ S_u^t = S_u^{t-1} + a_u \end{cases}$$

With the updated content written to the memory, subsequent calculations do not require any *read-write* operations in this time step. By repeating such a process, we activate the weight matrix to $\{0,1\}$ as $\Gamma_u = \sigma(S_u^t)$ and calculate the output $y = \Gamma_u \cdot S_c^t + (1-\Gamma_u) \cdot S_c^{t-1}$. After fully connected layer f_c and softmax, we can obtain the final classification result.

4) Backward Propagation and Parameter Update: For activation functions, the derivative can be calculated using a straight-through estimator. In the weight matrix, we use the straight-through estimator to get the gradient of the full-precision matrix and update W. Meanwhile, we can use the clip function to approximate the middle part of the f_t and further update the parameter Δ.

B. rEnforcer Design

rEnforcer is the dataplane design of RIDS. It is responsible for shaping the traffic based on model inference results. We depict the architecture of rEnforcer in Figure 4. rEnforcer has two pipelines. The first termed *S-pipeline*, is responsible for storing and updating the per-flow state. The second termed *D-pipeline*, determines the forwarding action based on the inference result. Specifically, S-pipeline contains three tables: (i) the state table that maintains the hidden state for each flow; (ii) the weight table that holds the weight to calculate subsequent outputs; (iii) the forwarding table stores the per-flow forwarding action (such as "nop" and "drop") and it is initially empty. If a 5-tuple index is matched by the forwarding table, the corresponding packet is tagged with the action and forwarded directly to the output port of the D-pipeline. Otherwise, the packet traverses the feature extraction modules of the S-pipeline before forwarding. Packets entering the D-pipeline perform prediction and update the forwarding table accordingly.

1) Feature Collector: We perform one-hot encoding for each incoming packet. Towards this end, our compiler will construct a match-action table with 4 entries with *range match* to store the range of values in SRAM.

2) Matrix Processor: The rEnforcer scheduler maintains three matrices, W_c, W_u, and f_c with ternary weight (*i.e.,* $\{-1,0,1\}$) that can be stored on the switch ASICs. We use match-action to implement the calculation process and set the *default rule* to match all packets.

3) Table Design: There are three tables maintained in rEnforcer: state table, weight table, and forwarding table. Among them, the state and weight tables have a similar structure. The first two columns in both tables are the 5-tuple index and the counter that records the number of packets for each flow indexed by its 5-tuple. The third column of state table and weight table are hidden state and update weight, respectively. As for the forwarding table, each entry contains a 5-tuple index and action. The action essentially is metadata that belongs to the maintenance information of the packet.

4) Conditional Branch: To fulfill the activation function in *sigmoid* and *tanh*, we perform an element-wise operation (if-else branch) for the vector. For each element, we use *control flow* logic to match the range (λ_1 or λ_2) and use *modified action* to update the activated value.

5) Softmax: In the D-pipeline, we perform model inference for each flow and determine its action. In model inference, the softmax operation is executed after fully connected layer f_c.

IV. Evaluation

Testbed setup. Our physical testbed contains multiple hosts with 100 Gbps Mellanox ConnectX-5 network cards and a Tofino switch (Wedge 100BF-32X, has 32 100 Gbps ports). The hosts are running Ubuntu 20.04.1 LTS with Linux kernel 5.4.0. They all have Intel Xeon(R) E5-2620V3 CPU (2.10 GHz, 8 cores), and 64 GB of memory. The code of RIDS is available online https://github.com/Secbrain/RIDS/.

Datasets and settings. We use two intrusion detection datasets and a malware dataset. (i) IDS2017/2018 [1] include a series of intrusion attacks, and various legitimate traffic of human

TABLE I: Compare with baseline models.

	Methods	*Accuracy*	*Precision*	*Recall*	*F1-score*
IDS 17&18	GRU (offline)	99.25±0.16	99.89±0.14	98.56±0.23	99.22±0.18
	HyperVision	97.05±0.46	96.36±0.34	97.82±0.28	97.11±0.36
	Mousika	92.75±0.43	91.37±0.33	92.64±0.49	91.96±0.45
	Kitsune	97.25±0.31	98.46±0.25	96.02±0.36	97.22±0.34
	RIDS	99.03±0.22	99.87±0.19	98.31±0.25	99.09±0.23
	Methods	*Accuracy*	*Precision*	*Recall*	*F1-score*
USTC	GRU (offline)	99.43±0.11	99.55±0.08	99.22±0.17	99.31±0.15
	HyperVision	98.85±0.21	98.69±0.25	98.90±0.15	98.85±0.19
	Mousika	94.25±0.52	93.53±0.50	95.12±0.36	94.30±0.42
	Kitsune	98.38±0.28	98.71±0.20	98.01±0.40	98.33±0.31
	RIDS	99.15±0.17	99.50±0.10	98.82±0.23	99.16±0.15

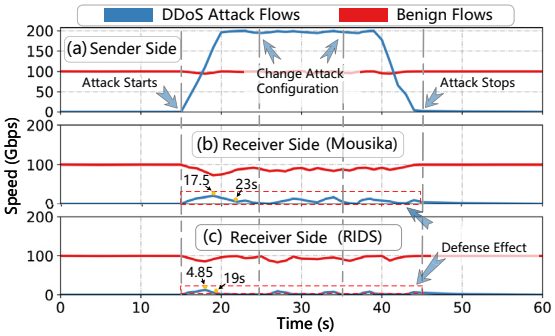

Fig. 5: DDoS mitigation evaluation.

interactions. (ii) USTC-TFC dataset [12] contains network traffic generated by malware and benign from real-world connections. About the hyperparameters for RIDS and the typical GRU, the maximum time step is 128, dimensions of input and hidden layer are both 4, learning rate is 0.001, batch size is 64, and the number of epochs is 100.

Compare with SOTA in offline. We first compare the detection effect of RIDS with state-of-the-art (SOTA) models and typical GRU. The results are summarized in Table I. Compared to a typical GRU, RIDS exhibits some accuracy drop due to the integer-weights design. The recall is from 98.56% to 98.31% in IDS 17&18, yet the slight precision drop (∼0.02%) means RIDS does not introduce mass false positive. For the deployable models (into switches), RIDS significantly outperforms Mousika in these detection metrics, *e.g.,* the IDS *Acc* of RIDS is ∼6.2% higher than Mousika. This is because RIDS considers the stateful RNN that could capture contextual semantics of traffic, which is lacking in stateless Mousika.

Online DDoS defense. We launch the diverse DDoS to saturate a 300 Gbps bandwidth. Figure 5(b) and (c) show how Mousika and RIDS prevent DDoS attacks at the receiver side. We find that Mousika exhibits $\{S^{DDoS}_{max} = 17.5, S^{DDoS}_{t=23} = 0\}$. However, the mitigation effect of RIDS is more effective, manifested as $\{S^{DDoS}_{max} = 4.85, S^{DDoS}_{t=19} = 0\}$, which can detect the attack flow earlier and reduce the peak value of DDoS traffic.

Overhead evaluations. The overhead results of RIDS system are collected from the real hardware switch using the P4 defense program. (i) Hardware utilization. In Table II, RIDS only uses 4.86% TCAM and 9.38% SRAM. Therefore, RIDS consumes a small number of hardware resources, which means RIDS is practical in current modern switches and is promising to extensibility. (ii) Throughput and latency. We build a 300 Gbps link and replay the five-day traffic of IDS2017. In Figure 6,

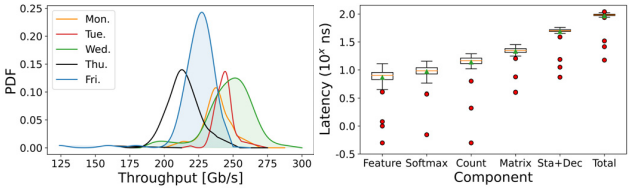

Fig. 6: Throughput and latency.

TABLE II: Hardware resource utilization breakdown in RIDS.

Modules	Switch Resource Usage Breakdown						
	Stage	xBar	Hash	Gateways	VLIW	SRAM	TCAM
Feature Collector	1	4	0	0	1	1	3
State Table+Decision	6	75	297	9	29	83	8
Conditional Branch	1	7	4	2	4	1	3
Matrix Processor	2	4	0	6	8	1	3
Softmax	1	1	8	3	3	0	0
Total	11	91	4.97%	10.94%	8.07%	9.38%	4.86%

RIDS realizes the real-time traffic detection and can achieve full bandwidth, *i.e.,* 300 Gbps. Moreover, RIDS program only introduces about 96 ns in total. This is acceptable, especially considering network RTTs (round-trip times) are typically at the order of milliseconds in the Internet core.

V. Conclusion

In this paper, we propose RIDS, a novel intrusion detection system that leverages the emerging hardware primitives to directly perform RNN model inference on the programmable switches. By employing a model and dataplane primitive co-designed approach, RIDS realizes the deep-learning-powered traffic classification capability inside the network with negligible overhead. We implement a prototype of RIDS and evaluate it extensively on our testbed. The results show that RIDS can achieve >99% F1 score for intrusion/malware detection, while only imposing nanoseconds of latency on the physical testbed with 300 Gbps maximum throughput.

References

[1] CICIDS. https://www.unb.ca/cic/datasets/.
[2] Diogo Barradas et al. Flowlens: Enabling efficient flow classification for ml-based network security applications. In *NDSS*, 2021.
[3] Pat Bosshart et al. P4: programming protocol-independent packet processors. *Comput. Commun. Rev.*, 44(3):87–95, 2014.
[4] Chuanpu Fu et al. Realtime robust malicious traffic detection via frequency domain analysis. In *CCS*, pages 3431–3446. ACM, 2021.
[5] Chuanpu Fu et al. Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis. In *NDSS*, 2023.
[6] Fengfu Li et al. Ternary weight networks. *CoRR*, abs/1605.04711, 2016.
[7] Hongda Li et al. vnids: Towards elastic security with safe and efficient virtualization of network intrusion detection systems. In *CCS*, 2018.
[8] Chang Liu et al. Fs-net: A flow sequence network for encrypted traffic classification. In *INFOCOM*, pages 1171–1179. IEEE, 2019.
[9] Zaoxing Liu et al. Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric ddos attacks with programmable switches. In *USENIX Security Symposium*, 2021.
[10] Yisroel Mirsky et al. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *NDSS*. The Internet Society, 2018.
[11] D. Project. Dpdk: Data plane development kit. [EB/OL], 2010. http://dpdk.org/ Accessed November 27, 2020.
[12] Wei Wang et al. Malware traffic classification using convolutional neural network for representation learning. In *ICOIN*. IEEE, 2017.
[13] Guorui Xie et al. Mousika: Enable General In-Network Intelligence in Programmable Switches by Knowledge Distillation. In *INFOCOM*. IEEE, 2022.
[14] Menghao Zhang et al. Poseidon: Mitigating volumetric ddos attacks with programmable switches. In *NDSS*. The Internet Society, 2020.